



RAPPORT D'EXPÉRIENCE (REX)

Mastère Spécialisé Big Data · Grenoble École de Management / Grenoble INP – Ensimag ·
Promotion 2025-2026

Sortir le modèle du chemin critique

Un an à transformer des prototypes d'IA générative en produit dans
la plateforme de fidélisation d'Amadeus

amadeus

Entreprise	Amadeus IT Group — division Loyalty, Nice
Auteur	Charafeddine ACHIR — Alternant Développeur GenAI
Période en entreprise	Octobre 2025 – Octobre 2026
Tuteur entreprise	Aniss TCHEKRI
Tuteur école	Nicolas GAILLOT

Présentation du projet

La plateforme de fidélisation d'Amadeus (LCP) ne comportait aucune brique d'intelligence artificielle avant cette alternance. Recruté dans le cadre d'une initiative visant à augmenter le produit par l'IA générative, j'ai d'abord identifié, avec mon tuteur, les cas d'usage où un modèle de langage et agent IA apportaient de la valeur, puis conçu et développé en autonomie le premier d'entre eux : un assistant conversationnel de configuration, fondé sur une génération augmentée par récupération (RAG) au-dessus de la documentation produit, puis enrichi d'une base de connaissances générée depuis le code source, d'un appel d'outils interrogeant la configuration réelle de chaque programme et doté d'un module de comparaison entre enivrants. Ces prototypes, présentés à des clients et prospects pendant cinq mois, ont convaincu les parties prenantes et débouché sur la création d'une équipe dédiée de onze personnes réparties entre la France et la Turquie, chargée d'industrialiser deux cas d'usage. J'y contribue depuis comme développeur intégré à la chaîne de développement, sur un agent d'IA qui anticipe le motif d'appel d'un membre au centre de contacts et recommande l'action suivante. Ce rapport analyse ce que le passage du prototype au produit a transformé dans ma pratique professionnelle.

Mots-clés : Intelligence artificielle générative · Architecture logicielle · Gouvernance des données · Éco-conception

Sommaire

1. Introduction	3
2. Le travail des compétences, item par item	4
2.1 Item 1 : maîtrise des principales techniques du Big Data	4
2.2 Item 2 : élaboration et pilotage d'une stratégie de gouvernance des données... ..	5
2.3 Item 3 : accompagnement des métiers de l'entreprise dans les projets Big Data	6
2.4 Item 4 : pilotage des hommes et des équipes multidisciplinaires.....	7
2.5 Item 5 : maîtrise des enjeux juridiques et éthiques autour de l'utilisation des données.....	7
3. Trois situations, regardées de près.....	8
3.1 Cas n°1 : quand la bonne réponse n'est pas dans la documentation	8
3.2 Cas n°2 : soixante secondes retirées d'un budget.....	9
3.3 Cas n°3 : onze personnes dans mon code	10
4. Bilan global et axes de progrès.....	11
4.1 Synthèse	11
4.2 Deux axes de progrès	12
5. Une proposition pour la transition écologique et sociétale.....	12
5.1 Une sobriété obtenue par accident	12
5.2 Un indicateur, et deux arbitrages qu'il rendrait décidables.....	12
5.3 Assister sans déqualifier	13
6. Conclusion	13
Bibliographie	14
Annexes	15
Annexe A : le panneau d'analyses tel qu'il apparaît à l'agent.....	15
Annexe B : extrait des tickets résolus dans l'équipe	15
Annexe C : documents de conception disponibles.....	16

1. Introduction

Amadeus IT Group fournit l'infrastructure technologique d'une large part de l'industrie du voyage : distribution aérienne, systèmes de réservation, solutions hôtelières et aéroportuaires. La division dans laquelle s'est déroulée mon alternance LCP, la plateforme sur laquelle des compagnies aériennes exploitent leurs programmes de fidélité : membres, points, statuts, transactions. Une même instance sert plusieurs compagnies, ce qui impose que chaque requête indique explicitement de quel programme elle parle, contrainte apparemment anodine qui structure, comme on le verra, une bonne partie des choix d'architecture.

Lorsque j'ai pris mon poste en octobre 2025, cette plateforme ne comportait aucune brique d'intelligence artificielle. Mon collègue alternant et moi avons été recrutés dans le cadre d'une initiative destinée à changer cela. Le mandat était volontairement ouvert : identifier où l'IA produirait de la valeur, le démontrer, et convaincre. Cette ouverture est la caractéristique la plus déterminante de mon alternance, parce qu'elle m'a placé successivement dans deux situations professionnelles très différentes.

D'octobre à février, j'ai travaillé en autonomie quasi complète. Après un travail de cadrage mené avec mon tuteur, un cas d'usage m'a été confié : un assistant conversationnel pour le module de configuration que j'ai spécifié, architecturé et développé seul. En parallèle, nous avons présenté ces prototypes à des clients et à des prospects. Cette phase s'est conclue par une décision structurante : les preuves de concept ont convaincu les parties prenantes, au moment où l'un des plus gros clients de la division demandait précisément à travailler sur des cas d'usage d'IA qu'il avait lui-même identifiés.

À partir de mars, une équipe dédiée a été constituée pour industrialiser ces prototypes : un product owner, un architecte logiciel senior en supervision, deux ingénieurs en CDI à temps plein, deux alternants, trois ingénieurs à mi-temps et deux stagiaires, répartis entre la France et la Turquie. Deux cas d'usage ont été retenus : Member Insight et le chatbot de configuration avec priorité au premier.

Mon travail a alors changé de nature : d'auteur unique d'un prototype, je suis devenu un contributeur parmi d'autres à une base de code partagée, avec des tickets, des revues, des tests et une chaîne de déploiement.

Ce rapport prend cette bascule comme fil directeur. La question qui l'organise est la suivante : qu'est-ce que le passage du prototype au produit transforme réellement dans le métier de développeur en IA générative ? Ma réponse, développée dans les pages qui suivent, est que la difficulté se déplace, elle quitte le terrain de la faisabilité technique, où le modèle fait l'essentiel du travail, pour celui de la fiabilité, de la latence, de la gouvernance des données et de la maintenabilité, où le modèle n'est plus qu'un composant parmi d'autres.

Le rapport présente d'abord le travail des blocs de compétences du référentiel (partie 2), puis analyse trois situations professionnelles en détail (partie 3), avant un bilan global assorti d'axes de progrès (partie 4). La partie 5 propose une analyse critique et une proposition relative à la transition écologique et sociétale de l'entreprise d'accueil, dans le cadre de l'engagement de GEM comme société à mission.

2. Le travail des compétences, item par item

Le référentiel du MS Big Data découpe le métier en cinq items. Je les reprends dans l'ordre : ce que j'ai fait, ce que j'ai livré, ce que j'en ai appris.

2.1 Item 1 : maîtrise des principales techniques du Big Data

Mon premier livrable a été un chatbot, et je tiens à préciser ce que ce mot ne veut pas dire. Il ne s'agit pas d'un modèle branché sur une boîte de texte. Le problème posé était le suivant : les équipes d'implémentation d'Amadeus configurent des règles métier complexes (accumulation de points, facturation, campagnes, transitions de statut) dont la documentation vit dans Confluence et se cherche mal. Résultat : paramétrage lent, erreurs fréquentes.

La réponse technique s'appelle RAG, pour *Retrieval-Augmented Generation*, et le principe tient en une phrase : on n'attend pas du modèle qu'il sache, on lui donne à lire. Toute la difficulté est dans le pipeline qui prépare la lecture, et c'est lui que j'ai construit. Les pages sont extraites de Confluence par API REST, le HTML est nettoyé, les images sont converties en descriptions textuelles par un modèle multimodal, sans quoi un schéma d'architecture devient un trou dans la base de connaissances. Vient ensuite le *chunking*, l'étape où le document est découpé en fragments : j'ai écrit un découpage contextuel qui préserve la hiérarchie des sections, parce qu'un paragraphe arraché à son titre parent devient ambigu et remonte mal. Chaque fragment est enfin transformé en *embedding*, un vecteur de plusieurs milliers de dimensions où la proximité géométrique approxime la proximité de sens, puis stocké dans un *vector store* Azure. À l'exécution, la question de l'utilisateur subit le même traitement, et les fragments les plus proches remontent pour être injectés dans le contexte du modèle.

Le service est *stateless* : chaque requête ouvre un *thread* neuf et l'historique de conversation voyage dans la charge utile, maintenu côté client. Ce choix simplifie le déploiement et évite de stocker des conversations côté serveur, soit un problème de conformité en moins.

La version 2 a introduit le *function calling*, ou *tool use* : le modèle ne répond plus seulement à partir de textes, il peut décider d'appeler une fonction que je lui ai déclarée. J'ai exposé un outil qui interroge la configuration réelle du programme et renvoie la valeur effective, ses critères et ses métadonnées, avec des codes de statut explicites pour chaque façon d'échouer. Le cheminement complet est raconté au cas n°1. Aussi ce chatbot été doté d'un module de comparaison de configuration entre différents environnements de déploiement (DEV, UAT, PDT, PROD) : une tâche qui avant été exécutée manuellement, grâce au chatbot se faisait en un rien de temps.

Sur Member Insight, j'ai manipulé une autre famille de techniques, celle des architectures multi-agents. Le runtime interne d'Amadeus expose un serveur A2A (*Agent-to-Agent*) en JSON-RPC : les agents se parlent par un protocole de messages plutôt que par des appels de fonction. Les appels à la plateforme, eux, sont packagés en serveur MCP (*Model Context Protocol*, le standard émergent pour exposer des outils à un modèle) : dix-neuf outils encapsulent l'API REST de loyalty. Le reste relève du software engineering classique appliqué à un contexte : un *fan-out* de dix-sept appels en parallèle avec un *timeout budget* par appel, un cache Redis par session à durée de vie courte, un *prefetch* en tâche de fond pour diminuer la latence perçue du dashboard et de douze mois d'historique de transactions pour que la première question de l'utilisateur trouve les données déjà chaudes dans le chat.

Dernière technique, et la plus déroutante au départ : comment tester un système dont la sortie est de la prose ? On n'écrit pas une assertion d'égalité sur une phrase. Le projet combine environ cent soixante-dix tests unitaires sur la moitié déterministe et un harnais conversationnel en *LLM-as-a-judge* : un modèle joue l'agent de centre d'appels, tient une vraie conversation multi-tours avec la pile déployée, et cinq modèles juges notent la transcription, appels d'outils internes compris. L'un des juges ne vérifie qu'une seule chose, et c'est la plus importante : l'assistant a-t-il affirmé un fait qui ne figure dans aucun résultat d'outil ? C'est le contrôle d'hallucination.

Preuves : documents de conception « AI Implementation – Configuration Module » et sa version 2 ; rapport fonctionnel et technique de Member Insight ; tickets LPATRAIN-51730 et 51731 (définition des agents et des prompts), 52432 (prefetch et cache de l'historique), 52440 (performance du bootstrap).

2.2 Item 2 : élaboration et pilotage d'une stratégie de gouvernance des données

La question qui a le plus changé ma façon de voir la gouvernance ne ressemblait pas du tout à une question de gouvernance. Elle ressemblait à un bug.

Le chatbot de configuration donnait parfois des réponses fausses. Pas des hallucinations : des réponses fidèlement recopiées d'une documentation qui, elle, était fausse. Les pages Confluence avaient été écrites à un moment donné, le code avait continué d'évoluer, et rien dans le système ne signalait l'écart. C'est ce qu'on appelle du *doc drift*, et c'est un problème de gouvernance de la donnée déguisé en problème de contenu. La bonne question n'est pas « comment corriger ces pages ? », c'est « quelle est la source de vérité, et qui garantit qu'elle l'est ? ».

J'ai donc proposé un changement de processus plutôt qu'une correction. La base de connaissances n'est plus rédigée : elle est générée depuis le code source, dont un script extrait les définitions de paramètres, leurs critères, leurs attributs et leurs descriptions, vers cinq formats de sortie : un schéma structuré pour l'outillage, des fiches unitaires pour la recherche sémantique, une référence consolidée, une version lisible par un humain, un format d'embedding. La documentation devient un artefact de *build*, régénéré avec le code. Elle ne peut plus diverger de ce qu'elle décrit, puisqu'elle en est dérivée. C'est cette bascule, d'un document maintenu à la main vers une *single source of truth* qui est le code lui-même, que je considère comme ma contribution la plus durable au produit.

Le deuxième sujet de gouvernance est arrivé avec Member Insight, et il se formule ainsi : quand on branche un modèle sur un système de production faisant foi, au nom de qui lit-il ? La réponse par défaut, celle qui va le plus vite, est un compte de service aux droits larges. Nous avons tranché dans l'autre sens : le jeton de l'utilisateur connecté est propagé jusqu'à la plateforme, et l'agent ne détient aucun privilège propre. Il voit exactement ce que voit la personne qui l'utilise, ni plus ni moins. Techniquement, c'est un détail d'implémentation. Du point de vue du management, c'est ce qui fait que l'arrivée de l'IA n'ouvre aucune nouvelle surface de droits et que le modèle d'habilitation existant reste celui qui gouverne. Aucune politique d'accès n'a eu à être réécrite.

Vient enfin la traçabilité, sans laquelle rien de tout cela ne se vérifie. Un identifiant de corrélation est généré à la porte d'entrée si l'appelant n'en fournit pas, propagé à chaque saut interne, envoyé à la plateforme en en-tête et estampillé sur chaque ligne de journal. Les journaux partent en JSON structuré vers Splunk, indexés par environnement, de sorte qu'un appel est reconstituable de bout en bout. C'est du *data lineage* appliqué non pas à des tables, mais à des décisions.

Preuves : script de génération de la base de connaissances depuis le code source ; ticket LPATRAIN-52344 (suppression du décodage des données personnelles côté MCP) ; ticket LPATRAIN-50599 (déplacement des identifiants de membre et de session vers les métadonnées du protocole, en remplacement d'une extraction par expression régulière).

2.3 Item 3 : accompagnement des métiers de l'entreprise dans les projets Big Data

C'est par cet item que tout a commencé, et dans les conditions les plus brutes qui soient : personne ne m'a remis d'expression de besoin, parce qu'il n'y en avait pas. Il fallait aller la chercher.

Avec mon tuteur, nous avons d'abord identifié le cas du chatbot configuration. Côté implémentation, le constat s'est révélé mesurable : configurer un programme suppose de naviguer dans une documentation dispersée, ce qui coûte du temps et produit des erreurs. Côté centre de contacts, le besoin est venu dans un second temps de la part d'un client, et le constat était réel. Quand un membre appelle, l'agent qui décroche ouvre son dossier et obtient tout : profil, soldes, statut, historique, notes, réclamations, commandes, bons. Chaque information est exacte, chaque information est sur un écran différent. Ce qu'il n'obtient pas, c'est une lecture de la situation. Ce membre est-il sur le point de perdre son statut ? Un vol de la semaine dernière n'a-t-il jamais été crédité ? A-t-il déjà réclamé deux fois pour la même chose ? Répondre suppose de cliquer pendant que le client attend. En pratique, l'agent demande « en quoi puis-je vous aider ? » et reconstruit la situation à partir de la conversation. Trente secondes et onze onglets : le besoin tient dans cette image, et elle parle autant au développeur qu'au directeur commercial.

J'ai formalisé le premier cas d'usage dans un document de cadrage qui tient lieu de cahier des charges : contexte fonctionnel, énoncé du problème, solution proposée, expérience utilisateur, spécification de l'interface, modèle de données, diagrammes de séquence.

Il y a aussi une dimension de modèle d'affaires que je n'avais pas anticipée et qui a fini par contraindre l'architecture. La fonctionnalité est fermée derrière un *feature flag* par compagnie et une permission par utilisateur : une compagnie qui n'a pas acheté la capacité ne la voit jamais. Ce n'est donc pas une amélioration diffuse du produit, c'est une capacité vendable client par client. Cette forme commerciale impose la multi-tenancy stricte décrite plus haut et interdit le raccourci qui consisterait à figer un programme dans le déploiement pour aller plus vite.

Reste la partie la moins technique et la plus décisive : défendre la valeur. D'octobre à février, avec Amine (mon collègue alternant), nous avons présenté ces prototypes à des clients et à des prospects. L'exercice demande de traduire une architecture en bénéfice métier, d'assumer les limites du prototype sans les cacher (j'ai appris qu'annoncer une limite avant qu'on ne la découvre coûte beaucoup moins cher que l'inverse) et d'encaisser des objections de faisabilité. Ces démonstrations ont produit un résultat que je n'aurais pas osé viser en début d'année : **une décision d'investissement, et onze personnes affectées à un sujet qui n'existait pas six mois plus tôt.**

Preuves : document de cadrage du module de configuration ; démonstrations client et prospect entre octobre et février ; contribution à l'estimation des ressources nécessaires à l'industrialisation ; arbitrage de priorisation entre les deux cas d'usage.

2.4 Item 4 : pilotage des hommes et des équipes multidisciplinaires

Le 9 mars, j'ai perdu quelque chose que je n'avais jamais identifié comme un privilège : le droit de décider seul. C'est l'item sur lequel je suis le moins avancé, et autant le dire avant de le détailler : je suis contributeur dans cette équipe, très loin d'être responsable d'équipe.

Onze personnes, deux pays, des profils et des disponibilités hétérogènes : un product owner en France, un architecte logiciel senior en supervision, deux ingénieurs à temps plein, deux alternants, trois ingénieurs à mi-temps et deux stagiaires en Turquie. Le mode projet est agile : itérations, *backlog* affiné collectivement, tickets, revues croisées. J'ai résolu vingt-quatre tickets à ce jour.

Ce que je pilote réellement, c'est mon propre périmètre et les interfaces qui l'entourent : découper un sujet en tickets qu'un collègue peut prendre sans moi, écrire la conception avant le code pour que la discussion porte sur les intentions plutôt que sur des lignes déjà écrites, et faire des revues qui expliquent le pourquoi au lieu de signaler le comment. Avec des collègues en Turquie et des contributeurs à mi-temps, le travail est largement asynchrone, et cette contrainte m'a appris la seule règle qui vaille dans cette configuration : une décision qui n'est pas écrite n'existe pas pour la personne qui la découvrira douze heures plus tard.

J'ai également porté un point de méthode. En phase amont, plaider pour une conduite planifiée aurait été absurde : le besoin n'était pas stable, la faisabilité pas acquise, et le seul moyen de trancher était de construire. Le mode adapté était un pipeline d'innovation, fait de boucles courtes (prototype, démonstration, ajustement du périmètre), où chaque démonstration compte comme une étape de validité et pas seulement comme un rendez-vous commercial. À partir de mars, la même activité relevait clairement d'une conduite agile ticketée, parce que le besoin était stabilisé et que le coût principal était devenu la coordination. Savoir que ces deux régimes ne se pilotent pas de la même façon est le vrai acquis de cet item.

Une dernière chose, qui n'est pas du pilotage mais qui y ressemble : j'ai appris à traiter la dette technique comme un travail légitime et non comme un aveu. Supprimer un nommage interne devenu obsolète dans toute la base de code, signaler cinq outils de plateforme définis deux fois à l'identique dans le même module, documenter une configuration morte laissée par une étape supprimée : ce sont des contributions à la santé collective du code, et les prendre de sa propre initiative est une manière de peser dans une équipe quand on n'y a aucune autorité.

Preuves : vingt-quatre tickets résolus (annexe B) ; ticket LPATRAN-50853 (dette technique de nommage) ; documents de conception versionnés avec le dépôt ; note d'état documentant explicitement les écarts entre le code et la documentation publiée.

2.5 Item 5 : maîtrise des enjeux juridiques et éthiques autour de l'utilisation des données

Le système lit des données personnelles de clients de compagnies aériennes et atteint un système de production faisant foi. Autant dire que cet item n'a pas été une case à cocher en fin de projet : il a déterminé des choix d'architecture, et parmi les plus structurants.

- **Minimisation, appliquée au bon endroit.** Les dix-neuf outils d'accès ne laissent jamais passer une réponse brute de l'API. Chacun extrait les champs utiles, et une couche de *redaction* retire noms, civilités, adresses électroniques et postales, téléphones, numéros de documents d'identité, dates et lieux de naissance avant

que la charge utile n'atteigne un modèle. Cas plus subtil : quand une entrée de journal d'audit décrit la modification d'un champ sensible, ce sont les valeurs avant et après qui sont masquées, pas seulement le nom du champ. Et l'instantané renvoyé à l'interface porte une chaîne de remplacement à la place du nom du membre : le modèle n'apprend jamais qui il est, et n'en a aucun besoin pour classifier.

- **Garde-fous contre le *prompt injection*.** Chaque message utilisateur est encadré : un préambule fixe le rôle et le périmètre de l'assistant, une instruction finale les réaffirme après le texte, lequel est explicitement étiqueté comme non fiable. Si le texte entrant tente de redéfinir le rôle de l'assistant, l'instruction placée après lui rétablit l'original. Mais la vraie protection n'est pas textuelle : la liste des outils autorisés à chaque agent est déclarée en configuration, donc une attaque réussie ne donne pas accès à un outil que l'agent n'a jamais reçu. L'agent de conversation transactionnelle dispose de six outils d'activité en lecture seule et ne peut atteindre ni le profil ni l'écriture de notes, quelle que soit la manière dont on le manipule.
- **Transparence.** Toute surface d'IA porte une mention « AI GENERATED » et un avertissement indiquant que le contenu peut être inexact et doit être vérifié avant d'agir. Les recommandations qui dépendent de la politique commerciale de la compagnie portent un indicateur dédié, traduit en interface par une mise en garde explicite : le système déclare qu'il refuse de deviner une règle à laquelle il n'a pas accès. Ce point répond directement aux obligations de transparence du règlement européen sur l'intelligence artificielle pour les systèmes interagissant avec des personnes.

L'éco-conception relève du même item, et le cycle de vie d'une fonctionnalité d'IA générative n'a pas la forme qu'on lui prête d'ordinaire. Nous n'entraînons aucun modèle : l'impact n'est pas dans l'entraînement mais dans l'inférence à l'exécution, répétée à chaque ouverture de dossier membre. La chaîne effectuait initialement quatre appels de modèle, dont deux n'exerçaient aucun jugement. Leur suppression est racontée au cas n°2, et la partie 5 en tire une proposition d'indicateur.

Reste l'éthique, qui est le point où je suis le moins tranquille. La position de conception est nette : le système assiste un décideur humain, il ne décide jamais à sa place. Il lit, il ne modifie pas les données d'un membre de sa propre initiative. Mais le risque réel n'est pas là. Il est dans le biais d'automatisation : à force de voir une prédiction juste, l'agent humain cesse de la vérifier, et le jour où elle est fausse, l'erreur passe. Un badge ne suffit pas à traiter ce risque, et je propose en partie 5 de le traiter par la mesure.

Preuves : tickets LPATRIN-52344 (données personnelles) et LPATRIN-50849 (garde-fous de prompt) ; couche de masquage et sa couverture de tests ; surfaces de transparence de l'interface (annexe A).

3. Trois situations, regardées de près

J'ai retenu ces trois situations parce qu'elles m'ont chacune obligé à abandonner une idée que je croyais acquise. Même trame à chaque fois : la situation, ce que j'ai fait, ce que j'en ai compris, et le niveau que je revendique.

3.1 Cas n°1 : quand la bonne réponse n'est pas dans la documentation

La situation

Première version du chatbot de configuration : un RAG propre. La documentation vectorisée, la question de l'utilisateur qui remonte les bons fragments, un modèle qui

rédige. Ça marchait, les démonstrations passaient bien, j'étais content. Et une catégorie entière de questions recevait des réponses parfaitement rédigées et parfaitement inutiles. Un utilisateur qui demande « quel est l'âge minimum d'adhésion pour le canal web ? » ne veut pas la définition du paramètre. Il veut savoir que c'est dix-huit ans, pour son programme, aujourd'hui.

Ce que j'ai fait

J'ai fait deux changements d'architecture. Le premier, décrit à l'item 2, remplace la base de connaissances rédigée à la main par une base générée depuis le code source. Le second dote l'assistant d'un outil interrogeable en temps réel, déclaré au modèle en fonction calling : quand la question porte sur une valeur, le modèle appelle l'outil, l'outil exécute une requête sur la configuration effective du programme et renvoie la valeur, ses critères et ses métadonnées. Le point sur lequel j'ai le plus travaillé est la granularité des codes de statut renvoyés (« non configuré », « entrée vide », « critères manquants », « aucune correspondance », « erreur »), parce que c'est ce qui permet à l'assistant de dire « ce paramètre n'a pas de valeur pour votre programme » au lieu d'inventer quelque chose de plausible.

Ce que j'en ai compris

Le facteur limitant n'était ni le modèle ni la qualité de la recherche sémantique. C'était la nature même de la source. Un système de récupération répond à la question « que dit la documentation ? » ; mon utilisateur posait la question « quelle est ma configuration ? ». Aucune optimisation du premier ne pouvait répondre à la seconde. Depuis, avant d'investir dans le réglage d'un système, je vérifie d'abord que la question qu'il sait traiter est bien celle qu'on lui pose.

Deuxième enseignement, plus concret : la fiabilité d'un appel d'outil se joue dans les cas d'échec, pas dans le cas nominal. Et troisième, le plus désagréable : un prototype qui marche peut marcher pour de mauvaises raisons. Une démonstration réussie n'est pas une validation fonctionnelle.

Niveau revendiqué : acquis, en voie de maîtrise, sur la conception de systèmes RAG et d'assistants outillés, puisque j'ai conduit le cycle complet en autonomie, du cadrage à la version 2. En cours d'acquisition sur leur durcissement en production, comme le montre la limite d'authentification déjà mentionnée. Items 1 et 2.

3.2 Cas n°2 : soixante secondes retirées d'un budget

La situation

Member Insight n'a de valeur que si le panneau est à l'écran avant que la conversation ne commence. Un panneau qui arrive après quarante secondes n'aide personne : l'agent a déjà posé sa question ouverte, et le système est devenu une décoration. Or la chaîne initiale exécutait quatre étapes strictement en série, et l'une d'elles était un agent fondé sur un modèle dont le travail consistait à effectuer neuf appels de plateforme fixes, avec des paramètres fixes, et à recopier des champs fixes des réponses. De la plomberie de données déterministe, acheminée par un modèle de langage. Coût : deux complétions supplémentaires, un saut réseau entre applications, et neuf appels de plateforme effectivement sérialisés.

Ce que j'ai fait

Deux changements. L'agent de récupération a été remplacé par du code ordinaire, puis ses neuf appels ont été fusionnés dans le même fan-out parallèle que les huit appels de l'instantané. Le timeout budget de la chaîne de démarrage, c'est-à-dire le plafond

garanti, celui qui décrit à quel point une mauvaise journée peut être mauvaise, est passé de cent sept à quarante-sept secondes. Chaque appel reste plafonné individuellement, et un échec dégrade ce champ vers une valeur par défaut au lieu de faire tomber la requête : la dégradation est par champ, pas par requête. Seules les deux données qui définissent le membre, qui il est et ce qu'il détient, ont le droit de faire échouer l'opération entière. J'ai également contribué au prefetch en arrière-plan de l'historique de transactions, pour que la première question de l'utilisateur trouve les données déjà en cache.

Ce que j'en ai compris

C'est de ce cas qu'est sorti le principe qui structure aujourd'hui ma façon de concevoir : un modèle de langage se paie sur le jugement. Pondérer plusieurs signaux imparfaits les uns contre les autres, écrire une phrase qu'un humain va lire : voilà ce qu'il fait mieux que du code. Pour un travail dont les règles peuvent s'écrire, c'est un choix médiocre, lent et cher. Toute étape énonçable en règles a donc été déplacée dans le code, et les deux qui relèvent réellement du jugement, classifier le motif d'appel et formuler la recommandation, sont restées au modèle. La même migration avait déjà été appliquée à l'instantané, qui était un agent et est devenu une fonction.

Niveau revendiqué : maîtrisé sur cette catégorie d'optimisation, qui consiste à repérer une étape de modèle injustifiée, la remplacer par du code, paralléliser des accès, raisonner en dégradation. Items 1 et 5.

3.3 Cas n°3 : onze personnes dans mon code

La situation

Jusqu'en février, j'étais l'auteur unique de mon code. Une décision d'architecture prenait le temps de la prendre. Une convention était celle que je m'étais donnée. Un raccourci ne coûtait qu'à moi. À partir de mars, la même activité s'exerçait dans une équipe de onze personnes sur deux pays, avec un product owner qui arbitre et un architecte qui supervise. La bascule a été plus dure que prévu, et pas pour des raisons techniques.

Ce que j'ai fait

J'ai adopté les pratiques de la chaîne existante plutôt que de négocier des exceptions : découpage en tickets traçables, revue de code dans les deux sens, tests écrits avec le code et non après, documentation de conception versionnée avec le dépôt. J'ai aussi traité de la dette technique de ma propre initiative, ce qui est la façon la plus simple de rendre service à des gens qu'on ne croise pas.

Le point sur lequel j'ai le plus progressé est cependant la manière de documenter les écarts. La documentation publiée décrit un routage des conversations entre deux agents selon des mots-clés ; le code, lui, envoie tous les messages à l'agent spécialiste des transactions, la fonction de routage existant et étant testée mais n'étant branchée nulle part. Il aurait été facile d'aligner la documentation sur l'intention. J'ai contribué à faire l'inverse : une note d'état qui suit le code, dit explicitement où le code et la documentation divergent, et donne l'impact fonctionnel de l'écart.

Ce que j'en ai compris

Le critère de qualité est passé de « ça marche » à « ça se maintient », et ce déplacement change ce qu'il est raisonnable de faire. Une astuce qui fait gagner une journée au prototypeur peut coûter une semaine à l'équipe six mois plus tard. Ce

raisonnement ne m'était pas naturel ; je l'ai appris en revue de code, en devant justifier des choix que je n'aurais jamais eu à expliciter seul.

J'ai aussi compris que la transparence sur ce qui n'est pas fini est une compétence professionnelle et pas un aveu de faiblesse. Une liste explicite de limites connues, avec leur impact, permet à l'équipe de décider ; une documentation optimiste ne protège personne, et surtout pas l'utilisateur.

Niveau revendiqué : acquis sur l'ingénierie logicielle collaborative, puisque je contribue au rythme de l'équipe et je prends des travaux de fond de ma propre initiative. En cours d'acquisition a acquis sur l'animation technique : j'ai porté une décision d'architecture devant l'équipe et défendu un arbitrage contesté jusqu'au bout (la suppression de l'agent pre-intent detection). Items 3 et 4.

4. Bilan global et axes de progrès

4.1 Synthèse

Item du référentiel	Principales preuves du travail réalisé	Autonomie	Niveau revendiqué
Item 1. Maîtrise des principales techniques du Big Data	Pipeline RAG complet (extraction, chunking contextuel, embeddings, vector store) ; fonction calling ; architecture multi-agents A2A et serveur MCP à 19 outils ; fan-out parallèle, cache et prefetch ; ~170 tests unitaires et harnais LLM-as-a-judge à 5 juges	Totale puis partagée	Acquis, en voie de maîtrise
Item 2. Gouvernance des données alignée sur la stratégie	Base de connaissances générée depuis le code source (single source of truth) ; propagation du jeton utilisateur plutôt que compte de service ; isolation multi-tenant sans valeur par défaut ; traçabilité de bout en bout par identifiant de corrélation	Élevée sur la proposition, partagée sur la mise en œuvre	Acquis
Item 3. Accompagnement des métiers dans les projets Big Data	Caractérisation du besoin sur une plateforme sans IA ; cahier des charges du module de configuration ; proposition et développement de POC ; formulation de la création de valeur ; démonstrations client et prospect ayant produit une décision d'investissement	Élevée	Maîtrisé sur la valorisation, acquis sur le cadrage
Item 4. Pilotage des hommes et des équipes multidisciplinaires	Mode agile : backlog, itérations, 24 tickets résolus, revues croisées ; travail asynchrone France-Turquie ; préconisation argumentée de deux modes projet selon la phase ; prise d'initiative sur la dette technique	Partagée, sans responsabilité hiérarchique	Acquis
Item 5. Enjeux juridiques et éthiques, éco-conception	Couche de rédaction des données personnelles au niveau des outils ; garde-fous prompt injection et cloisonnement des outils ; routage fail-closed ; transparence en interface ; 2 appels de modèle sur 4 retirés du chemin nominal ; diagnostic de deux vulnérabilités	Partagée	Acquis

Le déséquilibre est assumé et lisible. Mes niveaux les plus élevés sont sur les items 1 et 3, parce que la première phase de l'alternance m'a laissé seul face à la technique et face aux clients. L'item 4 est mon point faible et le restera tant que je serai contributeur plutôt que responsable de périmètre. L'item 5 est solide côté juridique et éco-conception, moins abouti côté éthique : j'ai identifié le risque de biais d'automatisation, je n'ai pas encore mis en place de quoi le surveiller.

4.2 Deux axes de progrès

1. Monter en maturité sur la sécurité opérationnelle. Je conçois correctement la minimisation et le cloisonnement des privilèges, mais je reste faible sur la gestion des identités et des secrets en production : le jeton d'une heure de mon prototype et la vérification de certificats désactivée en sont les deux marqueurs. Objectif : maîtriser les mécanismes de managed identity et de rotation de secrets, et reprendre ces deux points comme travail explicite plutôt que comme limites documentées.

2. Prendre position plus tôt et plus fort. Le cas n°1 montre que j'ai construit un système techniquement correct avant d'avoir vérifié qu'il répondait à la question réellement posée.

5. Une proposition pour la transition écologique et sociétale

Grenoble École de Management est une société à mission, ce qui engage ses étudiants à regarder d'un œil critique les pratiques de leur entreprise d'accueil. Je prends le sujet que je connais le mieux et sur lequel j'ai des preuves de première main : ce que coûte l'IA générative que nous déployons, et ce qu'elle fait au travail de ceux qui l'utilisent.

5.1 Une sobriété obtenue par accident

Le voyage porte une empreinte environnementale dominée par le transport, ce qui produit un effet pervers pour un éditeur de logiciels : la consommation du système d'information paraît négligeable en comparaison, et cesse d'être discutée. L'IA générative change pourtant l'ordre de grandeur. Un appel de complétion n'a pas le coût d'une requête en base, et notre chaîne en effectuait quatre à chaque ouverture de dossier membre.

Le point important, c'est que nous avons déjà obtenu une réduction substantielle sans jamais la nommer ainsi. Remplacer l'agent de récupération par du code ordinaire, comme raconté au cas n°2, a supprimé deux complétions par démarrage et un saut réseau. La même migration avait été appliquée à l'instantané. Ce n'est pas marginal : deux inférences sur quatre retirées du chemin nominal d'un service appelé à chaque appel téléphonique dans un centre de contacts.

Ma critique ne porte donc pas sur l'absence d'efforts, mais sur l'absence de mesure. Cette réduction a été motivée par la latence et par le coût de licence, pas par un objectif de sobriété, et personne ne sait la traduire en énergie. Nous documentons des budgets de délai à la seconde près et pas un seul indicateur de consommation. Or on ne pilote pas ce qu'on ne mesure pas : tant que la sobriété n'est qu'un effet secondaire heureux de l'optimisation de latence, elle disparaîtra le jour où les deux objectifs entreront en conflit. Et ils entreront en conflit, par exemple le jour où il faudra choisir entre un modèle plus rapide et un modèle plus petit.

5.2 Un indicateur, et deux arbitrages qu'il rendrait décidables

Un indicateur, pour quelques heures de travail. Le projet estampille déjà un identifiant de corrélation sur chaque ligne de journal et envoie ses enregistrements

structurés vers Splunk. Y ajouter, par interaction, le nombre de jetons consommés en entrée et en sortie et le modèle utilisé demande quelques heures, et donne immédiatement une consommation *par dossier membre ouvert*, donc par unité de valeur et non par requête technique. Converti en énergie à l'aide de facteurs publics, cet indicateur entre directement dans le cadre du référentiel général d'écoconception des services numériques. Je propose de l'exposer au même titre que la latence dans le tableau de bord de l'équipe, avec un seuil d'alerte en cas de régression.

Premier arbitrage : brancher le classifieur déterministe. Le projet contient déjà une fonction qui reproduit en code l'échelle de priorité des seize catégories de motif d'appel, couverte par des tests, et que rien n'appelle : le chemin de démarrage passe par le modèle. Elle avait été écrite comme repli en cas d'indisponibilité. Avec un indicateur de consommation, la question s'inverse légitimement : sur quelle proportion des dossiers le classifieur déterministe donne-t-il le même résultat que le modèle ? Si cette proportion est élevée, l'exécuter en premier et ne solliciter le modèle que sur les cas ambigus supprimerait une inférence sur deux, réduirait la latence et augmenterait la reproductibilité. Trois bénéfices alignés, ce qui est rare. Sans mesure, la question ne peut même pas être posée.

Second arbitrage : instruire le coût du fan-out agressif. Les dix-sept appels de plateforme partent simultanément ; quand la règle d'*early stop* se déclenche parce que l'instantané est déjà concluant, neuf d'entre eux sont jetés. Ce coût a été accepté parce que l'arrêt anticipé est rare et que l'alternative coûterait un aller-retour supplémentaire à chaque requête ordinaire. Mais le comportement est derrière un commutateur d'environnement, donc réversible sans modification de code. Il ne manque que la fréquence réelle de l'arrêt anticipé pour trancher sur des données plutôt que sur une intuition. La même instrumentation la fournit.

5.3 Assister sans déqualifier

L'effet social direct de ce système porte sur le travail des agents de centre de contacts, un métier régulièrement cité parmi les plus exposés à l'automatisation. La position de conception mérite d'être soulignée parce qu'elle n'allait pas de soi : le système assiste, il ne décide pas. Il lit, il ne modifie rien de sa propre initiative, et les recommandations qui dépendent de la politique commerciale de la compagnie sont marquées comme telles et renvoient l'agent à la procédure en vigueur.

Le risque résiduel n'est donc pas le remplacement, mais la déqualification silencieuse. À force de voir une prédiction juste, l'agent cesse de la vérifier, et le jour où elle est fausse, l'erreur passe intacte jusqu'au client. Je propose de traiter ce risque par la mesure plutôt que par la consigne : suivre le taux de non-suivi de l'action recommandée. Un taux nul ne voudrait pas dire que le système est parfait, mais que l'agent humain a cessé d'exercer son jugement : signal d'alerte, pas de réussite. L'indicateur a le mérite d'être doublement utile, puisqu'un taux élevé mesure inversement la pertinence réelle des recommandations, que nulle évaluation hors ligne ne capture. Il gagnerait à être complété par une formation courte portant sur ce que le système ne peut pas savoir : la politique commerciale du jour, le contexte de l'appel précédent, ce que le membre s'apprête à dire.

6. Conclusion

En un an, je suis passé d'une plateforme sans une ligne d'IA à un système déployé en plusieurs environnements derrière un feature flag, avec au passage la création d'une équipe de onze personnes née de prototypes que nous avons présentés. Ce trajet vaut

surtout par ce qu'il m'a fait traverser : le métier du prototypeur, seul juge de ses choix et évalué sur la démonstration, puis celui du contributeur, dont les choix sont discutés, testés, maintenus et parfois défaits.

La réponse que j'apporte à la question posée en introduction est que le passage au produit ne rend pas le travail plus difficile techniquement. Il déplace la difficulté. En prototype, le modèle fait presque tout et la seule question est « est-ce faisable ? ». En production, la question devient « à quelle vitesse, à quel coût, avec quelles données, au nom de qui, et que se passe-t-il quand ça échoue ? ». Et les meilleures réponses consistent le plus souvent à retirer le modèle de là où il n'a rien à faire. Le principe que je garde de cette année, à savoir qu'un modèle de langage se paie sur le jugement et non sur la plomberie, n'était pas une intuition de départ. C'est le résultat d'une chaîne que j'ai vue passer de cent sept à quarante-sept secondes de budget en remplaçant un agent par une fonction.

Mes acquis les plus solides sont sur la technique et sur l'accompagnement des métiers. Mes progressions les plus nettes restent à faire sur la mesure, c'est-à-dire instrumenter ce que j'optimise, sur la sécurité opérationnelle, et sur le fait de porter une position devant une équipe plutôt que de la démontrer seul dans mon coin. Ces trois points ne sont pas des faiblesses génériques : ce sont les trois endroits précis où, preuves à l'appui, mon travail s'est arrêté juste avant d'être complet. C'est aussi ce qui rend crédible à mes yeux la proposition de la partie 5, puisqu'elle consiste exactement à instrumenter ce que nous avons amélioré sans le mesurer, et à en faire un levier de sobriété assumé plutôt qu'un heureux hasard.

Bibliographie

- Arcep, Arcom, DINUM (2024). Référentiel général d'écoconception de services numériques (RGESN). Paris.
- ADEME, Arcep (2023). Évaluation de l'impact environnemental du numérique en France et analyse prospective. Rapport de mission.
- Parlement européen et Conseil (2024). Règlement (UE) 2024/1689 établissant des règles harmonisées concernant l'intelligence artificielle. Journal officiel de l'Union européenne.
- Parlement européen et Conseil (2016). Règlement (UE) 2016/679 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (RGPD).
- Schön, D. A. (1983). *The Reflective Practitioner: How Professionals Think in Action*. New York : Basic Books.
- Kolb, D. A. (1984). *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs : Prentice Hall.
- Amadeus IT Group (2025-2026). Documentation technique interne : plateforme LCP, boîte à outils A3TK, référentiel d'API de fidélisation. Documents à diffusion restreinte.

Annexes

Ces documents permettent d'approfondir certains points sans être nécessaires à la compréhension du rapport. Ils sont issus de la documentation technique interne et relèvent de la clause de confidentialité de la page de garde.

Annexe A : le panneau d'analyses tel qu'il apparaît à l'agent

AI Insights

AI GENERATED

Tier Member requires 190 TP by 27 Jul 2027 to qualify for PLATINUM	Balance → 2,550 miles current → 975 miles pending ⚠ Miles expiring: 1,250 miles in 30 days (05 Sep 2026) 900 miles in 60 days (05 Oct 2026) 400 miles in 90 days (04 Nov 2026)	Activity Last activity on 06 Aug 2026
Intent Detection Ask about points expiring soon High Some of the member's points are due to expire in the next few weeks, so they are likely calling to understand expiry details or how to use them before they lapse.		Next Best Action Show Detailed Balance Summary Review the member's current, pending, and soon-to-expire points and explain the expiry dates and rules. ① Policy-controlled recommendation. Verify current customer policy before execution.

Panneau affiché en haut du dossier membre à son ouverture : synthèse en trois volets (statut, soldes, dernière activité), motif d'appel prédit avec son niveau de confiance et sa justification, action recommandée. La mention « AI GENERATED », le bouton d'information portant l'avertissement d'inexactitude et la mise en garde « recommandation soumise à politique » sont les surfaces de transparence décrites à l'item 5.

Annexe B : extrait des tickets résolus dans l'équipe

LPATRAIN-52743	Fix pending/expiring balance
LPATRAIN-52727	[UI] Fix RandomUUID compatibility
LPATRAIN-52696	[Member snapshot] No last activity, despite activities returned
LPATRAIN-52687	[UI] Last activity date not aligned with last activity in transaction summary
LPATRAIN-52683	[UI] Change Tier sentence display
LPATRAIN-52644	[UI] Remove activity status from from the bootstrap
LPATRAIN-52643	[UI] Align chat between screens
LPATRAIN-52440	Performance enhancement of bootstrap with intent detection
LPATRAIN-52432	Prefetch transaction history at warmup and cache it
LPATRAIN-52431	Add details fields in transaction history response

	LPATRAIN-52397	Add the SAP to the called endpoint in the LUI + change logic for a3tk ses...
	LPATRAIN-52344	Remove personal data decoding in MCP
	LPATRAIN-52318	Implement prefetch of insights during security questions
	LPATRAIN-51731	Add agents and calls to data sources in A3TK
⋮		
	LPATRAIN-51730	Define required agents and prompts
	LPATRAIN-50853	[TECHDEBT] Remove "octomind" Naming from Codebase
	LPATRAIN-50849	[SEC] Add Prompt Guardrails
	LPATRAIN-50640	4. Add loyalty/tier specialist agent
	LPATRAIN-50599	1. Move member_id / session_id to A2A metadata, remove regex extracti...
	LPATRAIN-50561	Evaluate performance (response time of agent)
	LPATRAIN-50546	UI - Implement based on UX decision (prerequisite)

Échantillon des vingt-quatre tickets résolus à ce jour, dont plusieurs cités en preuve : garde-fous de prompt (50849), suppression du décodage de données personnelles (52344), performance du bootstrap (52440), prefetch et cache de l'historique (52432), définition des agents et des prompts (51730, 51731), dette technique de nommage (50853).

Annexe C : documents de conception disponibles

- « AI Implementation – Configuration Module » : contexte fonctionnel, proposition de solution, diagramme de séquence, modifications d'API, modèle de données, diagramme d'ensemble du premier prototype.
- « V2 of AI Implementation – Configuration Module » : génération de la base de connaissances depuis le code source, architecture de function calling, définition et structure de réponse de l'outil temps réel, diagrammes de composants et de classes.
- « Member Insight AI Agent, Functional & Technical Report » : rapport complet du second système (architecture, chaîne de démarrage, prédiction du motif d'appel, confidentialité et sécurité, assurance qualité, état courant et limites connues).